

1) Sistemas Operativos – Generalidades

a) ¿Quien es el encargado de administrar el tiempo de la CPU y cual es su función principal dentro del sistema?

El tiempo en la CPU lo administra el scheduler (planificador), pero se pueden indicar las siguientes funciones para el planificador dentro de un RTOS:

1. Asignación del tiempo de CPU: decide que tarea se ejecuta en cada momento.
2. Prioridades: Utiliza un algoritmo de planificación basado en prioridades.
3. Preemptividad: Interrumpe tareas de menor prioridad si una de mayor prioridad lo requiere.
4. Determinismo: Asegura que los tiempo de respuesta sean predecibles y consistentes.

b) Explique a que se denomina “Contexto de Ejecución”. ¿Cuales son las variables intervinientes?

Es el conjunto de información (Registro de CPU, Stack, Estado de la tarea, Recursos asignados) que utiliza el sistema para gestionar la ejecución de una tarea y poder reanudarla correctamente luego de una interrupción o cambio de contexto.

Podemos diferenciar en variables de hardware y variables de software. Las variables de hardware son gestionadas por el kernel del RTOS cuando se realiza el cambio de contexto, podemos resumir las variables de hardware en las siguientes; contador de programa, Registro de Estado, Registro de Propósito General, Puntero de Pila, Registros de Punto Flotante.

En cuanto a las variables de software nos referimos a las estructuras de datos para administrar el contexto, de las cuales podemos mencionar; Bloque de Control de Tarea (TCB), Tabla de Tareas del Planificador, Variables de Temporización.

c) ¿A que se denomina Sistema Operativo de Tiempo Real? ¿Por que usar un Sistema Operativo de Tiempo Real?

Es un sistema diseñado para aplicaciones donde el tiempo de respuesta es critico y debe ser predecible y deterministico. Esto garantiza que las tareas se ejecuten dentro de plazos estrictos. Se utiliza cuando se requiere precisión, fiabilidad y determinismo en sistemas donde las fallas no son aceptables, podemos resumir en las siguientes razones su uso:

- Garantiza respuestas en tiempos estrictos (Determinismo)
- Planificación predecible de tareas
- Baja latencia y respuesta rapida a interrupciones
- Eficiencia en hardware limitado
- Confiabilidad y estabilidad
- Soporte para concurrencia y comunicación entre tareas

2) Kernel de FreeRTOS

a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

Una tarea se define como una función en C, esto quiere decir que la definición es una plantilla de la cual podemos crear diferentes tareas de una misma definición. Podemos definir las siguientes características:

- La función que la define no devuelve parámetro y no recibe argumentos.
- Corre en un bucle infinito.
- La tarea necesita tener recursos asignados, estos pueden recuperarse eliminando la tarea.
- La tarea se crea usando la API `xTaskCreate`.
- Las tareas tienen prioridades numéricas siendo 0 la mas baja. Con estas prioridades el planificador decide cual ejecutar.
- Cada tarea tiene su propia pila (stack), el tamaño se indica al crear la tarea.
- Poseen estado; Running, Ready, Blocked, Suspend.
- Tienen sus mecanismo de comunicación y sincronización; Colas, Semáforos.
- Es critico el uso de **`vTaskDelay()`** para permitir que otras tareas se ejecuten.

```

5 void tarea1(void *pvParameters)
6 {
7     while (1)
8     {
9         printf("Ejecutando Tarea 1\n");
10        vTaskDelay(1000);
11    }
12 }
13 void tarea2(void *pvParameters)
14 {
15     while (1)
16     {
17         printf("Ejecutando Tarea 2\n");
18         vTaskDelay(1000);
19     }
20 }
21 int main() {
22     stdio_init_all();
23     xTaskCreate(tarea1, "Tarea1", 512, NULL, 1, NULL);
24     xTaskCreate(tarea2, "Tarea2", 512, NULL, 1, NULL);
25     vTaskStartScheduler();
26     while (true);
27 }

```

Se definen dos tareas; tarea1 de línea 5 a 12, tarea2 de línea 13 a 20. Dentro de main se usa la API para crear las tareas línea 23 y 24.

El ejemplo completo se puede ver en el ANEXO de Ejemplos, como Ejemplo 1.

Definición de una tarea y creación de una tarea

b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Una tarea posee 4 estados y transiciona entre estos estados:

- Running: indica que la tarea esta en ejecución.
- Ready: Indica que la tarea esta lista para entrar en ejecución, es decir en estado Running.
- Blocked: La tarea esta bloqueada, existen varias formas por las que se puede bloquear una tarea, las mas comun es por el uso de **`vTaskDelay()`**, pero también se puede bloquear por el uso de Colas, Semáforos o por que una tarea de mayor prioridad pasa al estado de ejecución.

- Suspend: Se utilizan APIs para suspenderla y volver a reanudarla. Si la tarea es suspendida debe reanudarse de forma manual, es decir se debe indicar un punto en donde la taera sea reanudada.

Una tarea no pasa de manera directa al estado Running, siempre debe estar en estado Ready para poder pasar al estado Running. Si la tarea esta en estado Blocked, esta pasara al estado Ready y luego a Running, una vez termine el evento que la bloquea. El camino se puede resumir en el siguiente imagen.

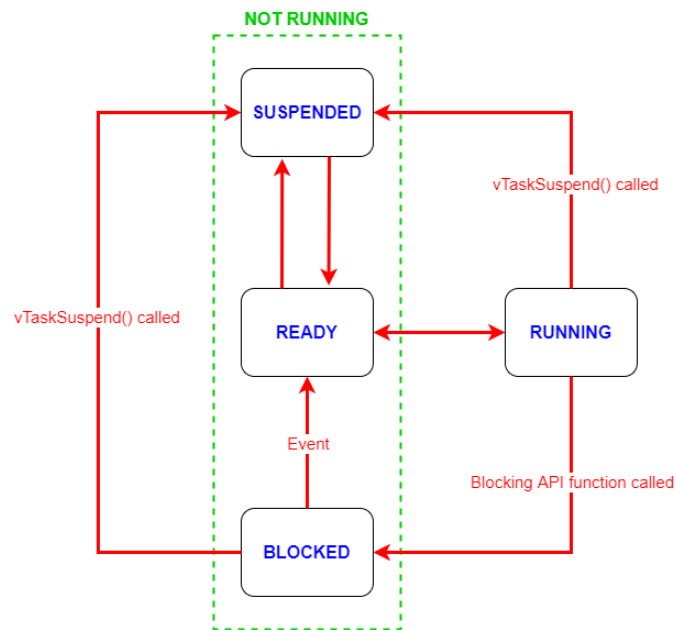


Figura 2. Estados de transición de una tarea

Ready → Running Transición: Lo maneja el planificador. Ocurre: Tarea entra en estado Blocked o Suspend Tarea de mayor prioridad pasa a Ready.	Running → Ready Transición: Forzada Ocurre: Tarea de mayor prioridad pasa a Ready. Tarea actual llama a taskYIELD().	Running → Blocked vTaskDealy xQueueReceive xQueueSend xSemaphoreTake xSemaphoreGive	Blocked → Ready Transición: automática Ocurre: Expira tiempo de delay. Llega dato a cola Semáforo disponible Evento esperado ocurre
Ready/Blocked/ Running → Suspend vTaskSuspend vTaskSuspendAll	Suspend → Ready vTaskResume: reanuda tarea. XtaskResumeFromISR: reanuda desde ISR xTaskResumeAll: reanuda planificador		

- La transición Blocked → Ready es automática cuando se cumple la condición de espera
- Suspended requiere intervención explícita del programador
- El planificador maneja Ready ↔ Running automáticamente
- Las ISRs pueden afectar estados con funciones terminadas en FromISR

c) ¿A que' nos referimos con el concepto de "tareas de procesamiento continuo"? ¿y a que nos referimos con el concepto de "tareas manejadas por eventos"?

Tarea de procesamiento continuo (TPC): Son tareas que, mientras están en el estado Running, realizan un trabajo activo y constante, consumiendo ciclos de CPU de forma continua hasta que completan su lógica o deciden ceder el control voluntariamente.

Tareas manejadas por eventos (TME): Son tareas que pasan la mayor parte de su vida en estado Blocked, esperando pasivamente a que ocurra un evento para ser despertadas. Solo consumen ciclos de CPU cuando realmente tienen trabajo que hacer.

Por ejemplo si tengo una tarea TPC que debe realizar alguna acción por presionar un boton la misma consumirá, de manera innecesaria, ciclos de clock a la espera de que se presione el boton. En su lugar se puede implementar una tarea que responda por medio una interrupción por hardware o que sea activada por otra tarea si se requiere.

d) ¿Cuales son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Una tarea puede bloquearse por los mecanismos que ofrece el RTOS para la sincronización y comunicación. Pero se pueden destacar lo siguientes eventos (los mas comunes):

- Delay: Se bloquea la tarea por un tiempo determinado, de manera que la tarea se ejecuta en forma periódica. Estos delay se implementan por APIs proporcionadas por el kernel.
- Evento de sincronización (semáforos binarios, contador y mutex): La tarea se bloquea a la espera de una señal proveniente de otra tarea o de una interrupción. Los semáforos tienen sus APIs.
- Eventos por comunicación (colas): La tarea se bloquea esperando recibir informacion, pero también se puede bloquear a la espera de enviar información. Se usan las APIs de colas, tales como xQueueSend y xQueueReceive.
- Otros eventos que se utilizan son Evento de agregación (Event Groups) y Evento directo (Notificaciones de tarea).

e) ¿Cual es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algun momento? Explicar.

La IDLE TASK es una tareas de prioridad cero que se crea automáticamente al llamar a vTaskScheduler(). Su única función es ejecutarse cuando no hay ninguna tarea util en estado Ready. Esta tarea nunca deben bloquearse ya que si se hiciera el sistema se quedaría sin nada que ejecutar, esto es así ya que el CPU siempre debe estar ejecutando instrucciones.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Las colas proveen un mecanismo de comunicación de tarea a tarea, de tarea a interrupción y de interrupción a tarea. Una cola puede almacenar un numero finito de datos, donde cada dato tiene un tamaño fijo es decir todos los datos son iguales. El máximo numero datos que almacena se conoce como longitud. La longitud de la cola y el tamaño de cada dato son indicados al crearse la cola. Las colas son usadas con un buffer de tipo FIFO, donde el primer dato que entra sera el primero en salir.

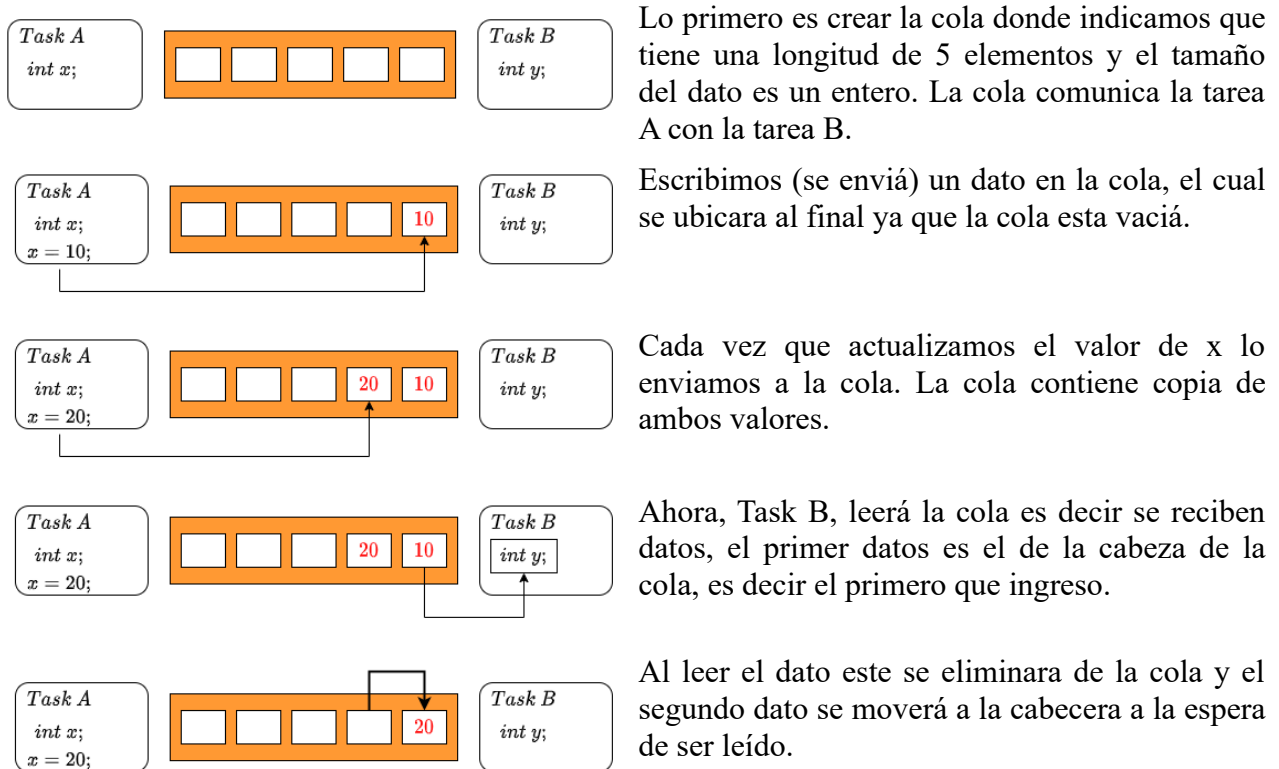


Figura 1. Lectura y escritura de una cola

Existen dos forma de implementar una cola:

1. Cola por copia: donde la cola copia el contenido de la variable y no el puntero. Esto quiere decir que al colocar un valor en la variable y enviarlo a la cola se copiara el valor, luego puedo cambiar el valor de la variable y volver a enviar el dato manteniendo el dato anterior.
2. Cola por referencia: Se almacena un puntero no una copia del dato, esta forma puede traer problemas de corrupción de datos.

Por defecto FreeRTOS utiliza Cola por copia.

Una cola puede realizar las siguientes acciones:

1. Puedes ser accedida por varias tareas.
2. Puede bloquear la tarea que intenta leerla si su contenido esta vacío. La tarea se bloquea hasta que llegue algún dato o finalice el tiempo de bloqueo, el cual se puede establecer. Si el tiempo finaliza y la cola sigue vacía se retorna un mensaje de error y se desbloquea la tarea.

3. Puede bloquear la tarea que intenta escribir si la cola esta llena. La tarea se bloque hasta que haya lugar en la cola o finalice el tiempo de espera. Si el tiempo finaliza y la cola sigue llena se retorna un error.
4. Puede bloquear mutiples tareas usando los dos casos anteriores.

Las colas pueden usarse como sincronización entre tareas pero su eficiencia no es buena si lo que se requiere es sincronización pura. Su principal propósito es la comunicación entre tareas. El uso de colas como medio de sincronización provocara que se utilice mas CPU y mas memoria ya que las misma almacenan datos. No son muy útiles cuando lo que se requiere es indicar que un evento esta listo.

4) Explique utilizando un ejemplo con código el porque es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

Si varias tareas escriben en una cola se tiene la ventaja de procesar varios datos en una sola tarea a la cual se le indica como es la lógica para tratar cada dato de la cola.

5) A la hora de enviar datos compuestos: explique como se lleva a cabo dicho proceso y por que no genera conflictos con la definición de la macro del recurso. ¿cual seria la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?.

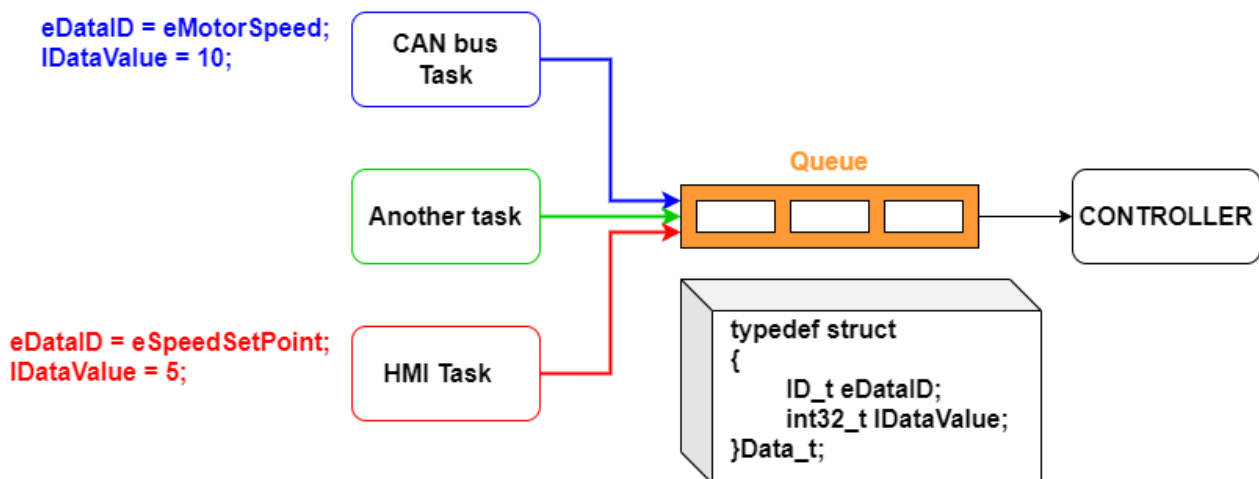
Con datos compuestos nos referimos a estructuras (struct) o uniones (union), lo cual es la forma estándar para enviar datos de diferentes tipos por medio de una cola. Para ello se crea una estructura que agrupe todos los campos de tipos de datos. Luego se crea la cola y se deberá usar *sizeof(estructura)* para indicar el tamaño de la estructura. Para enviar o recibir se pasa la dirección de memoria de la variable estructura a las APIs *xQueueSend()* y *xQueueReceive()*. El propio FreeRTOS se encarga de copiar todos los bytes de la estructura.

No existira conflicto ya que la API *xQueueCreate()* no necesita conocer la definición interna de la estructura. Solo debe saber la longitud de la cola y el tamaño en bytes de cada elemento. Esto es así ya que el kernel de FreeRTOS opera a nivel de bytes y no de tipos de datos, solo copiara un bloque de memoria de determinado tamaño desde el puntero origen al buffer de la cola.

Si los datos fueran de gran tamaño se utilizarían punteros, entonces lo que se almacenaría en la cola serian punteros a los datos. La ventaja radica en que los punteros tiene una longitud de alrededor de 4 u 8 bytes. Sustituimos la copia de datos, por la copia de punteros y se debe implementar una forma segura para gestionar estos punteros. La forma de implementar los punteros seria la siguientes:

- **Una cola de punteros libres:** Contiene punteros a buffers que están disponibles para ser usados.
- **Una cola de punteros llenos:** Contiene punteros a buffers que tienen datos listos para ser procesados.

6) Explicar el funcionamiento del esquema de la Figura 1.



En base a la estructura podemos decir que:

int32_t : Es el dato que enviá la tarea para que se procese

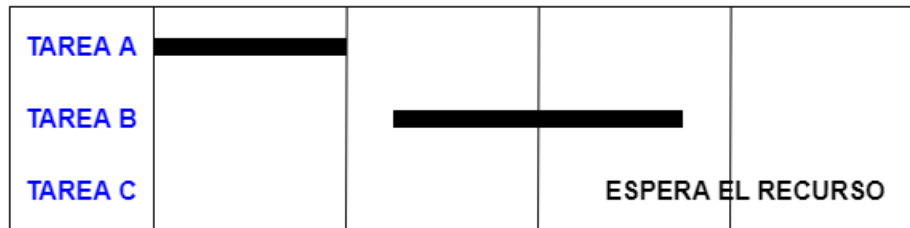
ID_t : Es el dato para identificar de que tarea proviene el dato a procesar

Se tienen diferentes tareas que envían datos a una cola, mientras que otra tarea leerá esos datos y los procesara según su origen. Para realizar esto se crea una estructura como la mostrada en la Figura 1 (debajo de la Queue), es esa estructura tenemos dos campos; uno indicara un valor específico (*IDataValue*), el otro sera para indicar de que tarea se trata (*eDataID*). De esta manera con el dato *eDataID* la tarea que lee sabrá como procesar cada dato que tome de la cola.

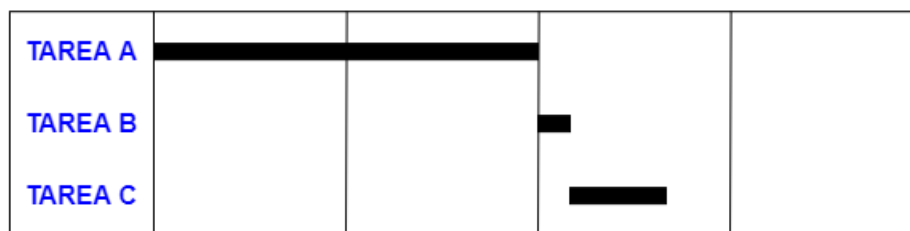
7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de semáforo mutex

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.



La tarea A se ejecuta sin problemas y toma el mutex, luego entra en ejecución la tarea B, que es de mayor prioridad que la tarea A, la tarea B se bloquea por un delay. Pero la tarea C que es de mayor prioridad no se ejecuta por que el recurso no esta disponible. Como consecuencia la tarea C de alta prioridad nunca se ejecuta.



La tarea A se ejecuta toma el mutex y hereda la prioridad de la tarea C. La tarea B esta a la espera (no se ejecuta), cuando el mutex es liberado la tarea B se ejecuta pero el mutex es tomado rapidamente por C y esta pasa a ejecutarse.

10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador